in Nanda & Lieberum (2022). Mean-ablations remove the information that *varies* in the reference distribution (e.g. the value of the name outputted by a head) but will preserve *constant* information (e.g. the fact that a head is outputting a name).

In this work, all knockouts are performed in a modified p_{IOI} distribution called p_{ABC}. It relies on the same generation procedure, but instead of using two names (IO and S) it used three unrelated random names (A, B and C). In p_{ABC}, sentences no longer have a single plausible IO, but the grammatical structures from the p_{IOI} templates are preserved.

We chose this distribution for mean-ablating because using p_{IOI} would not remove enough information helpful for the task. For instance, some information constant in p_{IOI} (e.g. the fact that a name is duplicated) is removed when computing the mean on p_{ABC}.

When knocking out a single node, a (head, token position) pair in our circuit, we want to preserve the grammatical information unrelated to IOI contained in its activations. However, the grammatical role (subject, verb, conjunction etc.) of a particular token position varies across templates. To ensure that grammatical information is constant when averaging, we compute the mean of a node across samples of the same template.

3 Discovering the Circuit

We seek to explain how GPT-2 small implements the IOI task (Section 2). Recall the example sentence "When Mary and John went to the store, John gave a drink to". The following human-interpretable algorithm suffices to perform this task:

1. Identify all previous names in the sentence (Mary, John, John).
2. Remove all names that are duplicated (in the example above: John).
3. Output the remaining name.

Below we present a circuit that we claim implements this functionality. Our circuit contains three major classes of heads, corresponding to the three steps of the algorithm above:

- *Duplicate Token Heads* identify tokens that have already appeared in the sentence. They are active at the S2 token, attend primarily to the S1 token, and signal that token duplication has occurred by writing the position of the duplicate token.
- *S-Inhibition Heads* remove duplicate tokens from Name Mover Heads' attention. They are active at the END token, attend to the S2 token, and write in the query of the Name Mover Heads, inhibiting their attention to S1 and S2 tokens.
- *Name Mover Heads* output the remaining name. They are active at END, attend to previous names in the sentence, and copy the names they attend to. Due to the S-Inhibition Heads, they attend to the IO token over the S1 and S2 tokens.

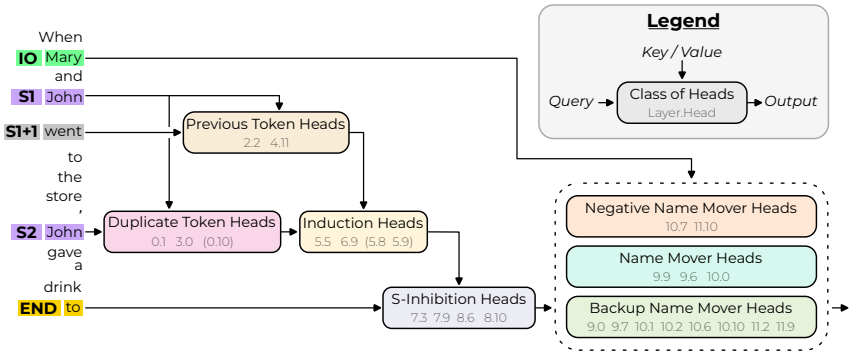


Figure 2: We discover a circuit in GPT-2 small that implements IOI. The input tokens on the left are passed into the residual stream. Attention heads move information between residual streams: the query and output arrows show which residual streams they write to, and the key/value arrows show which residual streams they read from.

A fourth major family of heads writes in the opposite direction of the Name Mover Heads, thus decreasing the confidence of the predictions. We speculate that these *Negative Name Mover Heads* might help the model "hedge" so as to avoid high cross-entropy loss when making mistakes.

There are also three minor classes of heads that perform related functions to the components above:

- *Previous Token Heads* copy information about the token S to token S1+1, the token after S1.
- *Induction Heads* perform the same role as the Duplicate Token Heads through an induction mechanism. They are active at position S2, attend to token S1+1 (mediated by the Previous Token Heads), and their output is used both as a pointer to S1 and as a signal that S is duplicated.
- Finally, *Backup Name Mover Heads* do not normally move the IO token to the output, but take on this role if the regular Name Mover Heads are knocked out.

In all of our experiments, we do not intervene on MLPs, layer norms, or embedding matrices, focusing our investigation on understanding the attention heads. In initial investigations, we found that knocking out each MLP individually led to good task performance, except for the first layer. However, knocking out all MLPs after the first layer makes the model unable to perform the task (Appendix J). A more precise investigation of the role of MLPs is left for future work.

Below we show how we discovered each of the seven components above, providing evidence that they behave as claimed. We found that it was easiest to uncover the circuit starting at the logits and working back step-by-step. Each step is divided into two parts. First, we trace the information flow backward from previously discovered components by finding attention heads that directly influence them; we do this using a technique we introduce called *path patching*, which generalizes the application of causal mediation analysis to language models introduced in Vig et al. (2020). Second, we characterize the newly identified heads by examining their attention patterns and performing experiments tailored to their hypothesized function.

3.1 Which heads directly affect the output? (Name Mover Heads)

Tracing back the information flow. We begin by searching for attention heads h directly affecting the model's logits. To differentiate indirect effect (where the influence of a component is mediated by another head) from direct effect, we designed a technique called **path patching** (Figure 1).

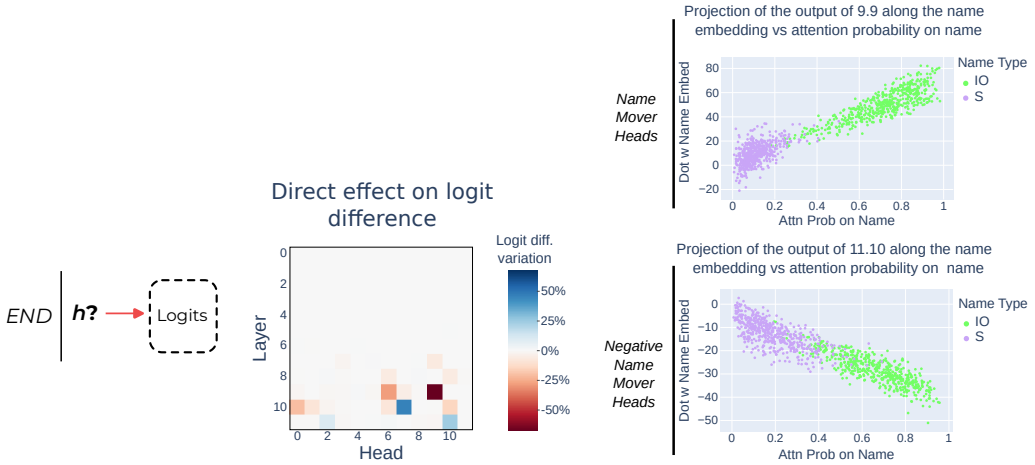
Path patching replaces part of a model's forward pass with activations from a different input. Given inputs x_{orig} and x_{new}, and a set of paths $\mathcal{P}$ emanating from a node h, path patching runs a forward pass on x_{orig}, but for the paths in $\mathcal{P}$ it replaces the activations for h with those from x_{new}. In our case, h will be a fixed attention head and $\mathcal{P}$ consists of all direct paths from h to a set of components R, i.e. paths through residual connections and MLPs (but not through other attention heads); this measures the counterfactual effect of h on the members of R. The layers after the element from R are recomputed as in a normal forward pass. See Appendix B for full details of the method.

We will always take x_{orig} to be a sample from p_{IOI}, and x_{new} the corresponding sample from p_{ABC} (i.e. where the names in the sentence are replaced by three random names). We run path patching on many random samples from p_{IOI} and measure how this affects the average logit difference. Pathways $h \rightarrow R$ that are critical to the model's computation should induce a large drop in logit difference when patched, since the p_{ABC} distribution removes the information needed to complete the IOI task.

To trace information back from the logits, we run path patching for the pathway $h \rightarrow$ Logits for each head h at the END position and display the effect on logit difference in Figure 3a. We see that only a few heads in the final layers cause a large effect on logit difference. Specifically, patching 9.6, 9.9, and 10.0 causes a large drop (they thus contribute positively to the logit difference), while 10.7 and 11.10 cause a large increase (they contribute negatively to the logit difference).

Name Mover Heads. To understand the heads positively influencing the logit difference, we first study their attention patterns. We find that they attend strongly to the IO token: the average attention probability of all heads over p_{IOI} is 0.59. We hypothesize that these heads (i) attend to the correct name and (ii) copy whatever they attend to. We therefore call these heads *Name Mover Heads*.

To verify our hypothesis, we design experiments to test the heads' functionality. Let W_U denote the unembedding matrix, and $W_U[IO]$, $W_U[S]$ the corresponding unembedding vectors for the IO and S tokens. We scatter plot the attention probability against the logit score $\langle h_i(X), W_U[N] \rangle$, measuring how much head h_i on input X is writing in the direction of the logit of the name N



(a) (b) (c)

Figure 3: (a) We are searching for heads h directly affecting the logits using path patching. (b) Results of the path patching experiments. Name Movers and Negative Name Movers Heads are the heads that have the strongest direct effect on the logit difference. (c) Attention probability vs projection of the head output along $W_U[IO]$ or $W_U[S]$ respectively. For S tokens, we sum the attention probability on both S1 and S2.

(IO or S). The results are shown in Figure 3c: higher attention probability on the IO or S token is correlated with higher output in the direction of the name (correlation $\rho > 0.81$, $N = 500$)[1].

To check that the Name Mover Heads copy names generally, we studied what values are written via the heads' OV matrix. Specifically, we first obtained the state of the residual stream at the position of each name token after the first MLP layer. Then, we multiplied this by the OV matrix of a Name Mover Head (simulating what would happen if the head attended perfectly to that token), multiplied by the unembedding matrix, and applied the final layer norm to obtain logit probabilities. We compute the proportion of samples that contain the input name token in the top 5 logits ($N = 1000$) and call this the copy score. All three Name Mover Heads have a copy score above 95%, compared to less than 20% for an average head.

Negative Name Mover Heads. In Figure 3b, we also observed two heads causing a large increase, and thus negatively influencing the logit difference. We called these heads *Negative Name Mover Heads*. These share all the same properties as Name Mover Heads except they (1) write in the opposite direction of names they attend to and (2) have a large *negative* copy score–the copy score calculated with the negative of the OV matrix (98% compared to 12% for an average head).

3.2 Which heads affect the Name Mover Heads' attention? (S-Inhibition Heads)

Tracing back the information flow. Given that the Name Mover Heads are primarily responsible for constructing the output, we next ask what heads they depend on. As illustrated in Figure 4a, there are three ways to influence attention heads: by their values, keys, or queries. In the case of Name Mover Heads, their value matrix appears to copy the input tokens (see previous section), so we do not study it further. Moreover, as the IO token appears early in the context, the Name Mover Heads' key vectors for this position likely do not contain much task-specific information. Therefore, we focus on the query vector, which is located at the END position.

We again use path patching, identifying the heads that affect each Name Mover Head's query vector. Specifically, for a head h we patch the path $h \rightarrow$ Name Mover Heads' queries (Figure 4a) and report

[1]In GPT-2 small, there is a layer norm before the final unembedding matrix. This means that these dot products computed are not appropriately scaled. However, empirically we found that approaches to approximating the composition of attention head outputs with this layer norm were complicated, and resulted in similar correlation and scatter plots.